Data Structures

BS (CS) _Fall_2025

Lab_05 Task

Learning Objectives:

1. Singly Linked List

Task 1: Generic Linked List Class using Templates in C++

Class Requirements

You need to design a singly linked list class using templates.

It should have:

- **Node<T>** class: contains
  - T data
  - Node<T>* next
- **LinkedList<T>** class: contains
  - Node<T>* head (pointer to first node)
  - int counter (tracks number of nodes)

Operations to Implement

Insertion Functions

- **insertAtHead(T item)**
  Insert a new node at the start of the list.
- **insertAtTail(T item)**
  Insert a new node at the end of the list.
- **insertAfter(T itemToInsert, T existingItem)**
  Find existingItem and insert itemToInsert after it.
- **insertBefore(T itemToInsert, T existingItem)**
  Find existingItem and insert itemToInsert before it.

Removal Functions

- **remove(T item)**
  Remove the node containing the given item (if found).
- **removeBefore(T item)**
  Find item and remove the node just before it.
  Example: {1 → 2 → 3 → 4}, removeBefore(3) → {1 → 3 → 4}
- **removeAfter(T item)**
  Find item and remove the node just after it.
  Example: {1 → 2 → 3 → 4 → 5}, removeAfter(3) → {1 → 2 → 3 → 5}

Utility Functions

- **isEmpty()**
  Returns true if the list has no elements.
- **search(T item)**
  Returns the pointer to node (or index-like position) if found, otherwise nullptr / -1.
- **print()**
  Prints all elements of the list.

- **reverse()**
  Reverses the list in-place.

Gtest :

```
// ---------- INSERTION TESTS ----------
TEST(LinkedListTest, InsertAtHeadEmpty) {
    LinkedList<int> list;
    list.insertAtHead(10);
    EXPECT_FALSE(list.isEmpty());
    EXPECT_EQ(list.search(10)->data, 10);
}

TEST(LinkedListTest, InsertAtHeadNonEmpty) {
    LinkedList<int> list;
    list.insertAtHead(10);
    list.insertAtHead(5);
    EXPECT_EQ(list.search(5)->data, 5);
}

TEST(LinkedListTest, InsertAtTailEmpty) {
    LinkedList<int> list;
    list.insertAtTail(20);
    EXPECT_EQ(list.search(20)->data, 20);
}

TEST(LinkedListTest, InsertAtTailNonEmpty) {
    LinkedList<int> list;
    list.insertAtHead(10);
    list.insertAtTail(30);
    EXPECT_EQ(list.search(30)->data, 30);
}

TEST(LinkedListTest, InsertAfterMiddle) {
    LinkedList<int> list;
    list.insertAtTail(5);
    list.insertAtTail(10);
    list.insertAtTail(30);
    list.insertAfter(25, 10);
    EXPECT_NE(list.search(25), nullptr);
}

TEST(LinkedListTest, InsertAfterNonExistent) {
    LinkedList<int> list;
```

```
    list.insertAtTail(5);
    list.insertAfter(100, 50); // should not insert
    EXPECT_EQ(list.search(100), nullptr);
}

TEST(LinkedListTest, InsertBeforeHead) {
    LinkedList<int> list;
    list.insertAtTail(5);
    list.insertBefore(1, 5);
    EXPECT_NE(list.search(1), nullptr);
}

TEST(LinkedListTest, InsertBeforeNonExistent) {
    LinkedList<int> list;
    list.insertAtTail(10);
    list.insertBefore(99, 50);
    EXPECT_EQ(list.search(99), nullptr);
}

// ---------- REMOVAL TESTS ----------
TEST(LinkedListTest, RemoveMiddle) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(5);
    list.insertAtTail(10);
    list.remove(5);
    EXPECT_EQ(list.search(5), nullptr);
}

TEST(LinkedListTest, RemoveHead) {
    LinkedList<int> list;
    list.insertAtTail(5);
    list.insertAtTail(10);
    list.remove(5);
    EXPECT_EQ(list.search(5), nullptr);
}

TEST(LinkedListTest, RemoveTail) {
    LinkedList<int> list;
    list.insertAtTail(5);
    list.insertAtTail(10);
    list.remove(10);
    EXPECT_EQ(list.search(10), nullptr);
}
```

```
TEST(LinkedListTest, RemoveNonExistent) {
    LinkedList<int> list;
    list.insertAtTail(5);
    list.remove(99);
    EXPECT_NE(list.search(5), nullptr); // unchanged
}

TEST(LinkedListTest, RemoveBeforeMiddle) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.insertAtTail(3);
    list.insertAtTail(4);
    list.removeBefore(3);  // removes 2
    EXPECT_EQ(list.search(2), nullptr);
}

TEST(LinkedListTest, RemoveBeforeHeadNoChange) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.removeBefore(1);  // nothing happens
    EXPECT_NE(list.search(1), nullptr);
}

TEST(LinkedListTest, RemoveAfterMiddle) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.insertAtTail(3);
    list.insertAtTail(4);
    list.removeAfter(2);  // removes 3
    EXPECT_EQ(list.search(3), nullptr);
}

TEST(LinkedListTest, RemoveAfterTailNoChange) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.removeAfter(2); // nothing happens
    EXPECT_NE(list.search(2), nullptr);
}

// ---------- UTILITY TESTS ----------
TEST(LinkedListTest, IsEmpty) {
```

```
    LinkedList<int> list;
    EXPECT_TRUE(list.isEmpty());
    list.insertAtHead(10);
    EXPECT_FALSE(list.isEmpty());
}

TEST(LinkedListTest, SearchFound) {
    LinkedList<int> list;
    list.insertAtTail(5);
    EXPECT_NE(list.search(5), nullptr);
}

TEST(LinkedListTest, SearchNotFound) {
    LinkedList<int> list;
    list.insertAtTail(5);
    EXPECT_EQ(list.search(50), nullptr);
}

TEST(LinkedListTest, ReverseList) {
    LinkedList<int> list;
    for (int i = 1; i <= 4; i++) list.insertAtTail(i);
    list.reverse(); // should be 4 → 3 → 2 → 1
    EXPECT_NE(list.search(4), nullptr);
    EXPECT_NE(list.search(1), nullptr);
}

TEST(LinkedListTest, ReverseEmpty) {
    LinkedList<int> list;
    list.reverse(); // should not crash
    EXPECT_TRUE(list.isEmpty());
}

TEST(LinkedListTest, ReverseSingle) {
    LinkedList<int> list;
    list.insertAtTail(10);
    list.reverse();
    EXPECT_NE(list.search(10), nullptr);
}

// ---------- EDGE & STRESS TEST ----------
TEST(LinkedListTest, LargeInsertAndRemove) {
    LinkedList<int> list;
    for (int i = 0; i < 1000; i++) list.insertAtTail(i);
    EXPECT_EQ(list.search(500)->data, 500);
    list.remove(500);
```

```
    EXPECT_EQ(list.search(500), nullptr);
}
```

Task 2: Detect and Remove Loop

Problem Statement

A loop (or cycle) occurs in a linked list when a node's next pointer does not point to nullptr but instead points back to a previous node in the list. This causes infinite traversal and must be fixed.

Your task is to extend your singly linked list class by implementing a function:

`bool detectAndRemoveLoop();`

The function should:

- Detect whether a loop exists in the linked list.
- If a loop exists, remove it by setting the last node's next pointer to nullptr.
- Return true if a loop was detected and removed, otherwise return false.

Input/Output Example

- **Input:**
  List = {1 , 2 , 3 , 4 , (points back to 2)}
- **Output after function call:**
  List = {1 , 2 , 3 , 4}

Gtest :

```
// 1. No loop in the list
TEST(DetectRemoveLoopTest, NoLoop) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.insertAtTail(3);

    EXPECT_FALSE(list.detectAndRemoveLoop());
    EXPECT_EQ(list.size(), 3);

    Node<int>* curr = list.head;
    int expected[] = { 1, 2, 3 };
    int i = 0;
    while (curr) {
        EXPECT_EQ(curr->data, expected[i++]);
        curr = curr->next;
    }
```

```
}

// 2. Tail points to head
TEST(DetectRemoveLoopTest, TailPointsToHead) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.insertAtTail(3);

    // create loop: tail -> head
    Node<int>* tail = list.head;
    while (tail->next) tail = tail->next;
    tail->next = list.head;

    EXPECT_TRUE(list.detectAndRemoveLoop());
    EXPECT_EQ(list.size(), 3);

    Node<int>* curr = list.head;
    int expected[] = { 1, 2, 3 };
    int i = 0;
    while (curr) {
        EXPECT_EQ(curr->data, expected[i++]);
        curr = curr->next;
    }
}

// 3. Tail points to middle (node 2)
TEST(DetectRemoveLoopTest, LoopBackToMiddle) {
    LinkedList<int> list;
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.insertAtTail(3);
    list.insertAtTail(4);

    Node<int>* curr = list.head;
    Node<int>* node2 = nullptr;
    while (curr) {
        if (curr->data == 2) node2 = curr;
        if (curr->next == nullptr) {
            curr->next = node2; // create loop
            break;
        }
        curr = curr->next;
    }
```

```
    EXPECT_TRUE(list.detectAndRemoveLoop());
    EXPECT_EQ(list.size(), 4);

    curr = list.head;
    int expected[] = { 1, 2, 3, 4 };
    int i = 0;
    while (curr) {
        EXPECT_EQ(curr->data, expected[i++]);
        curr = curr->next;
    }
}

// 4. Single node loop
TEST(DetectRemoveLoopTest, SingleNodeLoop) {
    LinkedList<int> list;
    list.insertAtTail(10);

    // create loop: node points to itself
    list.head->next = list.head;

    EXPECT_TRUE(list.detectAndRemoveLoop());
    EXPECT_EQ(list.size(), 1);

    EXPECT_EQ(list.head->data, 10);
    EXPECT_EQ(list.head->next, nullptr);
}
```